Optimisation de code de calcul scientifique

Les propagateurs acoustiques et élastiques de FUnTiDES comme cas d'étude

Guide de formation

13 octobre 2026

Table des matières

Chapitre 1 — L'optimisation de code : les concepts **3**

1.1 Qu'est-ce qu'optimiser ? . . . 3

1.2 Ce qu'on ne peut PAS optimiser : les invariants . . . 3

1.3 Où va le temps ? Toit de performance et intensité arithmétique . . . 4

1.4 Amdahl : pourquoi le noyau de faces commande la fête . . . 5

1.5 Exemple comparatif n° 1 : la fusion de noyaux (le gain réel) . . . 5

1.6 Exemple comparatif n° 2 : choisir le mapping parallèle (flat vs team) . . . 6

1.7 Exemple comparatif n° 3 : une optimisation illusoire, démasquée par le protocole . . . 6

1.8 Le paysage des transformations à modéliser . . . 6

Chapitre 2 — FUnTiDES et l'imagerie sismique **8**

2.1 Ce qu'est FUnTiDES . . . 8

2.2 Le métier : imagerie sismique et inversion . . . 8

2.3 Pourquoi le propagateur est le hot-spot . . . 8

2.4 Comment on pilote le code . . . 9

Chapitre 3 — La physique des propagateurs **10**

3.1 Le propagateur acoustique : une pression, deux paramètres . . . 10

3.2 Le propagateur élastique : trois déplacements, une loi de comportement . . . 10

3.3 Sources et récepteurs . . . 11

3.4 Conditions aux limites . . . 11

3.5 Atténuation : le sol visco-élastique . . . 11

3.6 Résumé comparatif . . . 12

Chapitre 4 — La méthode numérique **13**

4.1 Éléments spectraux et points de Gauss–Lobatto : pourquoi le schéma est explicite . . . 13

4.2 Le vecteur de rigidité : sum-factorization et GEMM . . . 13

4.3 Intégration temporelle : leapfrog amorti . . . 14

4.4 Variantes DG et DG-SEM : pourquoi de la Galerkin discontinue . . . 15

4.5 L'état adjoint et le gradient : le lien avec l'imagerie . . . 15

Chapitre 5 — HPC : Kokkos, GPU/CPU, MPI **16**

5.1 Kokkos : la couche de portabilité . . . 16

5.2 Deux politiques de parallélisation des éléments . . . 16

5.3 MPI et décomposition de domaine 17
5.4 Le contexte cluster : les conditions réelles d'exécution 17

Chapitre 6 — Le cas test et l'optimisation **18**
6.1 Le harnais : un instrument, jamais le sujet 18
6.2 Anatomie du cas test 18
6.3 Le protocole ratchet : quand un changement compte 19
6.4 Le paysage des axes d'optimisation 19
6.5 Construire et exécuter le cas test 20

Références **21**

Annexe A — Définitions des termes techniques **22**

Chapitre 1 — L'optimisation de code : les concepts

Ce chapitre expose les concepts de l'optimisation de code. Les chapitres suivants (2 à 5) sont la matière première — le métier, la physique, la méthode numérique, le matériel — et le chapitre 6 fixe le protocole de mesure. Chaque concept est ici illustré par un cas **réel** du code FUnTiDES, avec des exemples comparatifs, y compris un exemple d'optimisation **illusoire** qui a réellement été proposée et démasquée sur ce dépôt.

1.1 Qu'est-ce qu'optimiser ?

Optimiser un code scientifique, c'est appliquer au code source une **transformation** qui réduit un **coût mesuré** — ici, le temps moyen d'un pas de temps du solveur (`time_s`, sect. 6.2) — tout en respectant des **contraintes** :

- la **physique est inchangée** : le champ de sortie reste identique au champ de référence au sens des normes `l2_rel`/`linf_rel` (sect. 6.2) ;
- les **ressources tiennent** : l'empreinte mémoire (`mem_bytes`) n'explose pas ;
- le **build reste sain** : l'état de référence est un build sans `-O3` à cause d'un *use-after-free* latent (sect. 5.4) — une transformation qui fait crasher le build de référence n'est pas une optimisation.

Une transformation qui satisfait toutes les contraintes est dite **légale**. L'ensemble des transformations légales définit l'**espace de recherche**. La boucle de travail est donc : proposer une transformation légale candidate → la mesurer → ne la garder que si elle améliore le coût (le *ratchet*, sect. 6.3). C'est exactement cette structure — espace d'états, transformations légales, coût, contraintes — qu'une formalisation catégorielle se propose de modéliser (sect. 1.8).

1.2 Ce qu'on ne peut PAS optimiser : les invariants

Avant de chercher ce qu'on peut changer, il faut savoir ce qu'on ne peut pas toucher. Trois murs sont infranchissables sur ce cas test :

- **La condition CFL** (sect. 4.3) borne le pas de temps : le nombre de pas est fixé par la physique et la finesse du maillage. " Faire moins de pas de temps " n'est pas une transformation disponible — seule la durée d'un pas est optimisable.
- **La précision `float` (32 bits)** est le standard du domaine (sect. 5.2) : passer en `double` doublerait le trafic mémoire pour un bénéfice physique nul sur cette donnée — une anti-optimisation.
- **Le champ de sortie** doit rester l'expression de la même physique discrète. Simplifier un calcul en changeant sa sémantique (ordre d'un schéma, constante d'amortissement, application de l'éponge) est une modification de solveur, pas une optimisation.

Proposition	Verdict	Raison
Fusionner deux noyaux qui lisent les mêmes données	**légale**	même calcul, moins de trafic
Réordonner des additions flottantes	**légale** (à tolérance près)	l'écart est sous le seuil `l2_rel`
Sauter l'éponge sur les nœuds intérieurs	**illégale**	l'éponge vaut 1 à l'intérieur… mais il faut le *prouver*, pas le supposer
Augmenter le pas de temps au-delà de la CFL	**illégale**	la simulation diverge
Réduire l'ordre polynomial	**illégale**	change la physique discrétisée

La troisième ligne mérite une note : certaines transformations *paraissent* neutres mais échouent à la mesure — c'est la clause (2) du ratchet qui les attrape, pas le raisonnement.

1.3 Où va le temps ? Toit de performance et intensité arithmétique

Le concept quantitatif central : sur un GPU, un **noyau** — une fonction lancée massivement en parallèle sur le GPU, unité de travail de l'optimisation — est limité soit par le calcul (les **FLOPs**, *floating-point operations* : opérations arithmétiques en virgule flottante), soit par la **bande passante mémoire** (octets déplacés entre la **DRAM** — la mémoire principale du calculateur, l'étage lent que les caches ne font que masquer partiellement — et les cœurs de calcul). Le discriminant est l'**intensité arithmétique** :

$$IA = \frac{\text{nombre d'opérations flottantes}}{\text{octets déplacés depuis/après la DRAM}} \quad \text{(FLOP/octet)}$$

Chaque matériel a un **point de crête** — le rapport FLOPs-crête/bande-passante- crête (de l'ordre de la dizaine de FLOP/octet sur les GPU de calcul actuels). Un noyau dont l'IA est très en dessous du point de crête est **borné mémoire** : ajouter des FLOPs ne coûte presque rien, réduire les octets déplacés rapporte presque tout.

Exemple comparatif — la mise à jour leapfrog est bornée mémoire. Estimons l'IA du noyau `updateFieldsForward` (sect. 4.3, `sem_solver_impl.h`, lignes 1388–1519) par nœud et par pas, en acoustique sans atténuation :

- lectures DRAM : `mass_matrix`, `damping_matrix`, `taper_coeff`, champ précédent, champ courant, vecteur de travail → 6 flottants ;
- écritures DRAM : champ précédent, champ courant → 2 flottants ;
- trafic total ≈ 8 × 4 octets = **32 octets** (ordre de grandeur : les caches peuvent l'atténuer) ;
- opérations : le numérateur, le dénominateur, la division, deux produits par l'éponge ≈ **9 FLOPs**.

D'où $IA \approx 9/32 \approx 0{,}3$ FLOP/octet — deux ordres de grandeur sous le point de crête. Ce noyau est **entièrement borné mémoire** : la machine passe son temps à *attendre des octets*, pas à calculer. Toutes les transformations gagnantes sur ce code attaquent donc le trafic :

- **fusion** de noyaux : lire chaque donnée une fois au lieu de la relire à chaque noyau (sect. 1.5) ;
- **précision** `float` : trafic divisé par deux par rapport au `double` ;
- **scratch et cache** : réutiliser dans la mémoire partagée ce qui vient d'être lu (sect. 5.2) ;
- **compaction** : disposer les tableaux pour que les accès coalescents économisent des transactions mémoire.

Exemple comparatif — le chemin GEMM (produits matrice–matrice généralisés, développés en sect. 4.2) change de régime. L'action de rigidité acoustique existe en deux implémentations *de la même mathématique* (`sem_solver_impl.h`, lignes 342–347) : sum-factorization (boucles scalaires sur les points, IA modérée) et GEMM tensoriel (`Qk_Hexahedron_Tensorial.h` : les éléments regroupés en matrices, réutilisation au niveau des registres, IA bien plus élevée, délégation à `KokkosBlas`). Un même produit mathématique peut ainsi vivre des deux côtés du toit : c'est une **reformulation algébrique**, une classe de transformation à part entière.

1.4 Amdahl : pourquoi le noyau de faces commande la fête

La loi d'Amdahl borne le gain total d'une optimisation en fonction de la part du temps qu'elle touche. Si une fraction f du temps est accélérée d'un facteur s, le facteur d'accélération global vaut :

$$S = \frac{1}{(1-f) + \frac{f}{s}}$$

Sur le cas test (sect. 6.2), les sessions passées ont mesuré que le noyau de faces DG concentre **environ 60 %** du temps d'un pas, et que les fusions du membre de droite ont rapporté **environ 16 %** de gain. Amdahl en tire le décor de toute la suite :

- optimiser *parfaitement* tout sauf le noyau de faces : $f = 0{,}4$, $s \to \infty \to S \leq 1/0{,}4 = 2{,}5\times\ldots$ mais plus réalistement, doubler la vitesse du reste ($s = 2$) donne $1/(0{,}6 + 0{,}2) = 1{,}25\times$;
- doubler la vitesse du noyau de faces seul : $1/(0{,}4 + 0{,}3) \approx 1{,}43\times$ — le même effort dépensé sur le noyau dominant rapporte plus.

D'où la règle de méthode : **profiler avant d'optimiser**, attaquer d'abord la fraction dominante, et se méfier d'un gain local qui se dilue dans le global. Le harnais mesure le global (`time_s`) ; le profilage par noyau (fait lors des sessions passées) dit où le temps va.

1.5 Exemple comparatif n° 1 : la fusion de noyaux (le gain réel)

Un pas de temps du solveur déroule plusieurs lancements distincts — le fichier `sem_solver_impl.h` contient à lui seul 27 déclarations de `Kokkos::parallel_for` (précalculs, contributions, mise à jour…). Chaque lancement a un coût fixe (mise en file, synchronisation) et surtout **relit** ses entrées depuis la mémoire globale.

Algorithme 1 **Fusion de noyaux — avant : deux lancements séparés**

```
parallel_for("noyau A", n, ...);   // lit p, écrit work
parallel_for("noyau B", n, ...);   // relit p et work
```

Algorithme 2 **Fusion de noyaux — après : un lancement unique**

```
parallel_for("A+B fusionnés", n, ...);   // lit p une fois, fait A puis B
```

Deux gains se cumulent : le coût fixe de lancement disparaît et — c'est le principal sur un code borné mémoire (sect. 1.3) — la donnée partagée n'est lue qu'une fois. Deux instances réelles dans le dépôt :

- **noyau de faces DG** : amortissement de bord et flux d'interface calculés dans un seul `parallel_for` (`computeBoundaryDampingAndInterfaceFlux`, `dg_solver_impl.h`, ligne 196) ;
- **“ Fused Verlet ” DG-SEM** : la mise à jour temporelle des deux sous-domaines (DG et SEM) fusionnée en un seul lancement (`applyVerletFused`, `dg-sem_solver_impl.h`, lignes 273 et 302).

C'est par des fusions de ce type sur le membre de droite que les sessions passées ont obtenu leurs ∼16 % (sect. 6.3). **Contrepartie à connaître** : la fusion augmente la pression registre d'un noyau, donc peut réduire l'occupation (le nombre de blocs simultanément résidents) ; c'est le rôle des bornes de lancement (`LaunchBounds`, `sem_solver_impl.h`, ligne 371) que de négocier ce compromis — et seule la mesure tranche (sect. 6.3).

1.6 Exemple comparatif n° 2 : choisir le mapping parallèle (flat vs team)

Même mathématique, deux façons de répartir le travail — un choix de *mapping*, au cœur de la portabilité Kokkos (sect. 5.2) :

	Variante `_Flat`	Variante `_Team`
Politique	`RangePolicy` : 1 fil par élément	`TeamPolicy` : 1 équipe par élément
Données de travail	pile privée de chaque fil	mémoire **scratch** partagée par l'équipe
Synchronisation	aucune	barrières internes (`TeamThreadRange`)
Élégante pour	petit ordre (buffers réduits)	ordre élevé (forte réutilisation)
Dans le code	`computeElementContributions_Iso_Flat`, ligne 635	`computeElementContributions_Iso_Team`, ligne 741

Le dispatch est statique sur l'ordre polynomial : l'élastique d'ordre 1 prend le chemin plat (`kMaxOrderForFlatElastic=1`, `sem_solver.h`, ligne 221), les ordres élevés le chemin équipe — c'est une décision **dynamique à coût nul**, mais prise à la compilation : une extension d'optimisation pourrait vouloir l'explorer par cas.

Même famille — **calculer ou caché ?** : la géométrie d'un maillage cartésien est identique pour tous les éléments. La sum-factorization (sect. 4.2) propose donc deux variantes : recalculer la Jacobienne par élément (`computeElasticStiffnessSumFactTeam`, ligne 686) ou la sortir de la boucle élémentaire via la variante à géométrie constante (ligne 703) qui lit depuis la scratch des valeurs précalculées. C'est le compromis fondamental *arithmétique contre trafic mémoire*, dans un code borné mémoire : caché gagne presque toujours — mais encore une fois, la mesure le confirme.

1.7 Exemple comparatif n° 3 : une optimisation illusoire, démasquée par le protocole

La leçon la plus utile de l'historique de ce dépôt. Lors des sessions d'optimisation passées sur le DG-SEM, une piste a été explorée : **supprimer des barrières `Kokkos::fence()`** entre noyaux. Chronométrée côté hôte, la modification paraissait plus rapide — et elle a d'abord été prise pour un gain.

Elle n'en était pas un : le chronomètre hôte mesure quand le *flot CPU* reprend la main ; retirer une barrière libère simplement le CPU plus tôt pendant que le GPU continue — ou s'empire — en arrière-plan. Le travail GPU, lui, n'avait pas diminué. Le gain était un **artefact de mesure**. Les vrais gains de ces sessions sont venus des fusions du membre de droite (∼16 %, sect. 1.5). Trois morales, directement pertinentes pour une extension d'optimisation automatique :

1. **Une mesure vaut ce que vaut son protocole** : échauffement, GPU au repos, médiane de répétitions (sect. 5.4, sect. 6.3) ne sont pas du décor.
2. **Un optimiseur — humain ou algorithmique — peut être trompé** : une transformation qui " améliore la métrique " en dégradant la correction de la mesure est une régression déguisée. Le ratchet existe précisément pour cela.
3. La chronométrie doit se faire au plus près du travail mesuré : le harnais chronomètre `n` pas complets et divise, plutôt que de sommer des temps par noyau.

1.8 Le paysage des transformations à modéliser

Récapitulons le chapitre comme un espace de recherche :

- **Objets** : des variantes de code (le source étant la même sémantique physique discrète) ;
- **Transformations légales** — celles rencontrées dans ce chapitre : fusion de noyaux (sect. 1.5), changement de mapping parallèle (sect. 1.6), reformulation algébrique (sect. 1.3 : GEMM), placement/caching de données (sect. 1.6), staging hôte↔device, compaction mémoire (sect. 5.2) ;
- **Transformations de méthode** : celles qui changent le schéma discret lui-même (ordre polynomial, intégrateur temporel, conditions de bord, physique enrichie) et se jugent en précision/coût contre une référence convergée, plutôt qu'en identité (sect. 6.4) ;
- **Invariants** : le champ de sortie au sens `l2_rel`/`linf_rel`, la CFL, la précision, le build (sect. 1.2) ;

- **Coût** : `time_s` (et `mem_bytes` en contrainte) — mesuré par le harnais (sect. 6.2) ;
- **Ordre** : Amdahl (sect. 1.4) dit par où commencer.

Une extension à base de théorie des catégories /topos /C-sets peut voir les noyaux et leurs dépendances de données comme les objets d'une catégorie, les transformations légales comme des morphismes (avec leur composabilité — fusion après fusion), et les invariants comme des contraintes sur les objets — mais ce cours ne préjuge pas du formalisme : il fournit le terrain de jeu, ses transitions réelles, et l'instrument de mesure qui juge.

Chapitre 2 — FUnTiDES et l'imagerie sismique

2.1 Ce qu'est FUnTiDES

FUnTiDES — *Fast Unstructured Time Dynamic Equation Solver* (`README.md`, ligne 1) — est une bibliothèque de simulation de propagation d'ondes acoustiques et élastiques en 2D et 3D, résolues par une méthode d'éléments spectraux d'ordre élevé (SEM). L'interface utilisateur est en Python (bindings pybind11, module `pyfuntides`) ; le cœur de calcul, lourdement optimisé, est en C++ adossé à Kokkos pour la portabilité performante CPU/GPU (`README.md`, ligne 3).

Le dépôt expose quatre solveurs proxy sélectionnables à la configuration CMake (`cmake/ProxyConfig.cmake`) :

- **SEM** — éléments spectraux continus, le propagateur de référence (acoustique et élastique) ;
- **DG** — Galerkin discontinue avec pénalisation SIPG (acoustique) ;
- **DG-SEM** — solveur couplé : une couche DG absorbante autour d'un intérieur SEM (acoustique) ;
- **DG p-adaptatif** — ordre polynomial par élément (acoustique).

La profondeur d'instanciation des templates est contrôlée par `MAX_SOLVER_ORDER` /`MAX_DIFFERENTIATOR_ORDER` (ordres 1 à 9 ; `README.md`, ligne 36).

2.2 Le métier : imagerie sismique et inversion

Le cas d'usage moteur est l'**imagerie du sous-sol**. Le principe physique :

1. une **source** (explosion, vibreux) émet une onde dans le sol ;
2. l'onde se propage et se réfléchit/réfracte aux interfaces entre milieux de propriétés différentes (changements de vitesse de propagation) ;
3. des **récepteurs** en surface (hydrophones, géophones) enregistrent le signal arrivant au cours du temps : une **trace sismique** par récepteur ;
4. à partir d'un très grand nombre de traces (tirs × récepteurs), on reconstruit le **modèle du sous-sol** — les champs de vitesses v_p, v_s et de densité ρ — dont les réflexions expliquent les enregistrements observés.

La boucle " reconstruire le modèle " s'appelle l'inversion, et sa forme moderne la plus générale est la **FWI** (*Full-Waveform Inversion*) (Virieux & Operto, 2009) : on ajuste le modèle en minimisant l'écart entre traces simulées et traces observées,

$$J(m) = \frac{1}{2} \sum_{\text{tirs, récepteurs, temps}} \| \mathcal{R}\, p[m] - d_{\text{obs}} \|^2,$$

où m est le modèle (par ex. κ, b pour l'acoustique), $p[m]$ la solution de l'équation d'onde dans ce modèle, $\mathcal{R}$ l'opérateur qui échantillonne le champ aux récepteurs, et d_{obs} les données mesurées.

2.3 Pourquoi le propagateur est le hot-spot

Le point central : **chaque évaluation de J et de son gradient exige de résoudre l'équation d'onde en temps**. Plus précisément :

- une itération de FWI = pour chaque tir, **une propagation directe** (source → récepteurs, champ p) **plus une propagation adjointe** (résidus injectés aux récepteurs, champ rétropropagé, cf. sect. 4.5) ;
- une inversion réaliste, c'est des dizaines d'itérations × des centaines ou milliers de tirs : **des dizaines de milliers de propagations** du même solveur sur le même type de maillage ;

- le coût d'une propagation est lui-même un produit (nombre de pas de temps) × (coût d'un pas), le pas étant borné par la condition CFL (sect. 4.3) — on ne peut pas " sauter des pas ".

Tout gain sur le **coût d'un pas de temps du propagateur** se multiplie donc par des dizaines de milliers. C'est ce qui fait de ce code une cible d'optimisation représentative, et un cas test légitime pour évaluer une extension MIMIR qui choisit des transformations d'optimisation : les gains sont mesurables (temps par pas), et les erreurs sont détectables (le champ physique doit rester identique, sect. 6.3).

FUnTiDES n'implémente pas la boucle d'inversion elle-même : il fournit la brique *propagation* (champ direct, champ adjoint, gradient de la fonction coût vis-à-vis du modèle — `src/gradient/impl/acoustic/` et `src/gradient/impl/elastic/`), le reste de la chaîne (optimisation du modèle) appartenant à la couche appelante. La démonstration de bout en bout existe toutefois dans le dépôt : `examples/fe/qualitative_gradient_test.py` vérifie qualitativement le gradient, et `documentation/acoustic/formulation.md` sect. 3–4 documente la linéarisation de Born et l'état adjoint jusqu'aux expressions de gradient effectivement codées.

2.4 Comment on pilote le code

Deux points d'entrée :

- **Python** (interface principale) : les exemples `examples/fe/solver_cartesian.py` (mono-rang), `solver_cartesian_mpi.py` (MPI), `solver_multigpu.py` /`solver_multigpu_multinode.py` construisent un maillage cartésien, créent le solveur via la fabrique, allouent les champs et font tourner la boucle en temps ; le driver générique `python/run_funtides.py` lit une configuration YAML (`examples/example_config.yaml`) ;
- **C++** : le driver de référence `src/main/fe/src/sem_proxy.cc` (installé comme `build/bin/funtides-sem`) expose toutes les physiques et variantes en ligne de commande, et sert d'étalon : le harnais de benchmark (chapitre 6) reproduit volontairement ses choix de configuration pour rester comparable.

La validation de justesse passe par `makevalidate` (`README.md`, lignes 95–105) : des solutions calculées sont comparées à des **références analytiques** stockées dans `tests/validation/analyticalsolution/` (`P.dat` pour l'acoustique, `Ux.dat` pour l'élastique), via `tests/validation/validate_solution.cc`.

Chapitre 3 — La physique des propagateurs

Ce chapitre décrit la physique **telle qu'implémentée** dans FUnTiDES, symbole par symbole. Ce n'est pas un cours : c'est la lecture guidée des équations que le code résout, et des choix de formulation qui se répercutent jusque dans les noyaux GPU.

3.1 Le propagateur acoustique : une pression, deux paramètres

L'acoustique de FUnTiDES est une formulation scalaire en **pression**, du second ordre en temps, à densité variable, paramétrée en **module de compression** $\kappa = \rho\, v_p^2$ et **flottabilité** $b = 1/\rho$ (`documentation/acoustic/formulation.md`, sect. 1) :

$$\kappa \frac{\partial^2 p}{\partial t^2} \;-\; \nabla \cdot \big(b\, \nabla p \big) \;=\; f,$$

où p est le champ de pression et f le terme source. Le champ discrétisé ne comporte qu'une composante : `WavefieldAcoustic::kNumFields=1`, nommée `"pressure"` (`src/solver/fe/impl/acoustic/include/wavefield_acoustic.h`, lignes 15–18).

Pourquoi cette paramétrisation plutôt que (v_p, ρ) ? Parce que les matrices éléments finis qui en découlent sont **linéaires en leurs paramètres** : $M(\kappa)$ pour la masse, $R(b)$ pour la rigidité (sect. 4.1), si bien que la variation du problème pour une perturbation $\delta\kappa$, δb s'écrit avec les mêmes noyaux — c'est ce qui rend l'état adjoint et le gradient " gratuits " à implémenter (`formulation.md` sect. 3.1 : $\delta M(\kappa) = M(\delta\kappa)$, $\delta R(b) = R(\delta b)$). C'est un exemple typique de choix de formulation au niveau maths qui contraint toute la chaîne logicielle en aval.

3.2 Le propagateur élastique : trois déplacements, une loi de comportement

L'élastique est une formulation en **déplacement** $\mathbf{u} = (u_x, u_y, u_z)$, second ordre en temps : `WavefieldElastic::kNumFields=3` (`src/solver/fe/impl/elastic/include/wavefield_elastic.h`, ligne 17), avec `kNumRhsComponents=3` côté source (`rhs_elastic.h`, ligne 14) — un terme source peut porter des poids par composante, ce qui permet d'injecter un **moment tensor** (source élastique réalistique) et pas seulement une force ponctuelle.

La loi de comportement par défaut est **isotrope** (paramètres de Lamé λ, μ), avec les vitesses converties depuis (v_p, v_s, ρ) dans le noyau élémentaire :

$$\mu = \rho\, v_s^2, \qquad \lambda = \rho\,(v_p^2 - 2 v_s^2),$$

c'est-à-dire, en code (`src/solver/fe/impl/common/include/sem_solver_impl.h`, lignes 588–589) :

Algorithme 3 **Conversion des vitesses en paramètres de Lamé**

```
float mu = rho * vs * vs;
float lambda = rho * (vp * vp - 2.0f * vs * vs);
```

La contrainte de Cauchy s'assemble à partir du gradient de déplacement $H[i][t] = \partial u_t / \partial x_i$ (symétrisé en déformation) :

$$\sigma \;=\; \mu\,(H + H^{\mathrm{T}}) \;+\; \lambda\, \mathrm{tr}(H)\, I.$$

C'est exactement le contenu de `elasticFluxIso` (`src/solver/fe/impl/common/include/elastic_flux.h`, ligne 49) : calcul du gradient physique (`physicalGradient`, ligne 17), assemblage de σ symétrique, puis " pull-back " vers l'élément de référence (`pullBackStress`, ligne 29). Le fichier fournit aussi deux variantes anisotropes — utilisées quand la physique des roches exige que la vitesse dépende de la direction de propagation :

- **VTI** (*Vertical Transverse Isotropy*), `elasticFluxVti` (ligne 74) : 6 coefficients de rigidité indépendants en notation de Voigt ($c_{11}, c_{12}, c_{13}, c_{33}, c_{44}, c_{66}$) ;
- **TTI** (*Tilted Transverse Isotropy*), `elasticFluxTti` (ligne 100) : matrice de rigidité symétrique 6×6 complète, notation de Voigt avec cisaillements d'ingénieur.

La variante est choisie au runtime via `setAnisotropyType` (`sem_solver.h`, ligne 259) et dispatchée dans le calcul des contributions élémentaires (`sem_solver_impl.h`, lignes 196–212). Il existe enfin un solveur **acousto-élastique** couplant les deux physiques (`src/solver/fe/impl/acoustoelastic/`), hors du périmètre de ce cours.

3.3 Sources et récepteurs

- **Ondelette temporelle** : une source de Ricker (dérivée seconde d'une Gaussienne, le standard sismique), `evaluateRicker` et `computeSourceTerm` (`src/utils/include/utils.h`, lignes 8 et 47), d'ordre 0, 1 ou 2, paramétrée par la fréquence centrale `f0` et le pic `tpeak`.
- **Injection spatiale** : la source est ponctuelle dans un élément ; son amplitude est répartie sur les $(r+1)^3$ points de l'élément par des **poids d'interpolation de Lagrange** au point source (`ComputeRHSWeights`). Des poids naïfs exciteraient des modes non physiques et feraient diverger la simulation — le harnais de benchmark documente explicitement ce piège (sect. 6.2).
- **Récepteurs** : dans le driver de référence, les traces sont échantillonnées et écrites via ADIOS2 ; le gradient d'imagerie injecte le résidu aux mêmes positions (sect. 4.5).

3.4 Conditions aux limites

Une simulation réaliste se fait sur un domaine fini, alors que l'onde physique continue au-delà : il faut donc des frontières **absorbantes** qui laissent sortir l'onde, plus une éventuelle **surface libre** (le sol à l'air libre, où la pression s'annule et la contrainte élastique est nulle). FUnTiDES combine deux mécanismes :

1. **Matrice d'amortissement de bord** (type Rayleigh) : un terme de dissipation S n'est assemblé que sur les **faces externes** des éléments de bord (`computeDampingMatrix`, `sem_solver_impl.h`, ligne 1731). En acoustique, le coefficient scalaire vaut $\alpha = \sqrt{b/\kappa} = 1/(\rho\, v_p)$. En élastique, il est **directionnel par composante** (`sem_solver_impl.h`, lignes 1825–1832) :

Algorithme 4 **Coefficient d'amortissement de bord élastique, directionnel par composante**

```
real_t localIncrementx = aux * (velocityVp * fabs(nx)
                                + velocityVs * sqrt(ny * ny + nz * nz));
```

c'est-à-dire v_p selon la normale, v_s sur les directions transverses — l'onde P et l'onde S ne sortent pas à la même vitesse.

1. **Éponge en cosinus** : un profil d'atténuation multiplicatif `spongeTaperCoeff_` (`sem_solver.h`, ligne 329) appliqué à chaque pas de temps aux champs courant et précédent après la mise à jour (`sem_solver_impl.h`, lignes 1456–1457) — configurable dans le YAML (`boundaries:` dans `examples/example_config.yaml`, lignes 86–88).

La **surface libre** est traitée au niveau des nœuds dans le noyau de mise à jour : en acoustique, pression imposée à zéro (`sem_solver_impl.h`, lignes 1433–1435) ; en élastique, mise à jour non amortie spécifique aux nœuds de surface libre (lignes 1469–1491).

3.5 Atténuation : le sol visco-élastique

Les vraies roches dissipent l'énergie (les hautes fréquences s'atténuent plus vite). FUnTiDES offre une atténuation viscoélastique par **solide linéaire standard** (SLS) : des **variables de mémoire** supplémentaires par nœud et par

mécanisme (`attenuationMemoryVariables_`), évoluant par la récurrence du pas de temps (`sem_solver_impl.h`, lignes 1441–1452) :

$$\gamma = \frac{2 - \omega\, dt}{2 + \omega\, dt}, \qquad \mathrm{mem}^{n+1} = \gamma\, \mathrm{mem}^{n} + \beta\, \mathrm{travail}^{n},$$

où ω est la pulsation de référence du mécanisme et β son coefficient anelastique. Les paramètres qualité Q_p, Q_s viennent du YAML (`attenuation:` dans `example_config.yaml`, lignes 80–82). Chaque mécanisme SLS ajoute un état par nœud : c'est un exemple direct de la manière dont une décision physique se paie en mémoire et en bande passante au niveau HPC.

Avertissement — le PML n'est pas dans l'arbre de travail. Une frontière absorbante plus performante (C-PML, *convolutional perfectly matched layer* ; Komatitsch & Martin, 2007) existe dans l'histoire du projet : branches locales `feature/pml-acoustic-second-order`, `feature/pml-elastic-second-order` (et `feature/elastic-gemm`), scripts de build `scripts/build_sem_acoustic_pml_bench.sh` et `scripts/build_sem_elastic_pml_bench.sh`, test unitaire restant `tests/units/solver/impl/test_pml_coefficients.cc`. Mais **aucun code PML n'est présent dans `src/`** (le header `pml_coefficients.h` que le test inclut n'existe pas dans cette arborescence). Si vous cherchez " le PML de FUnTiDES ", vous ne trouverez que ces vestiges ; le schéma effectivement en place est celui du sect. 3.4.

3.6 Résumé comparatif

	Acoustique	**Élastique**
Inconnue	pression p (1 champ)	déplacement $\mathbf{u}$ (3 champs)
Paramètres	κ, b (ou v_p, ρ)	λ, μ (iso) ; 6 coeff. (VTI) ; 6×6 (TTI)
Ondes propagées	P seulement	P et S (couplées)
Coût par pas	rigidité scalaire : 1 gradient	rigidité vectorielle : 9 flux + loi de comportement
Damping de bord	scalaire $1/(\rho v_p)$	directionnel v_p/v_s par composante
Variante GEMM	oui (sect. 4.2)	non (sum-factorization uniquement)
Chemin dans le code	`src/solver/fe/impl/acoustic/`	`src/solver/fe/impl/elastic/`

Le coût élastique est structurellement plus lourd : trois champs signifient trois mises à jour temporelles, neuf flux de référence dans la sum-factorization (sect. 4.2), et une loi de comportement évaluée à chaque point de quadrature. C'est le même squelette de noyau pour les deux physiques — la classe est templatisée sur le trait de physique (`physics_traits_acoustic.h` /`physics_traits_elastic.h`), et le nombre de champs pilote les boucles internes.

Chapitre 4 — La méthode numérique

4.1 Éléments spectraux et points de Gauss–Lobatto : pourquoi le schéma est explicite

La méthode d'éléments spectraux (SEM) est une méthode d'éléments finis d'ordre élevé, conçue pour les problèmes d'onde (Komatitsch & Tromp, 2002 ; Komatitsch, Barnes & Tromp, 2000). Le domaine est découpé en **hexaèdres** ; sur chaque élément, le champ est développé sur une **base tensorisée de polynômes de Lagrange aux points de Gauss–Lobatto–Legendre (GLL)** d'ordre r (FUnTiDES génère $r = 1$ à 9 : `LagrangeBasis1.h` … `LagrangeBasis9GL.h`, inclus dans `src/discretization/fe/impl/makutu/include/Qk_Hexahedron_Lagrange_GaussLobatto.h`, lignes 10–18). Chaque élément porte $(r+1)^3$ points de quadrature/interpolation ; l'index local $lIdx = i + j(r+1) + k(r+1)^2$ se projette sur un nœud global via `globalNodeIndex(e,i,j,k)`.

La propriété décisive des points GLL — la raison pour laquelle tout le reste du solveur est simple et rapide — est la **collocation** : les nœuds d'interpolation et les points de quadrature coïncident. La matrice de masse élémentaire

$$M^e_{ij} = \int_{K^e} \kappa \, \phi_i \, \phi_j$$

devient alors **diagonale** (formulation *lumpée*), et la matrice de masse globale M aussi. Dans le code, M n'est jamais une matrice : c'est un simple vecteur diagonal assemblé par `computeGlobalMassMatrix` et stocké dans `massMatrixGlobal_` (`formulation.md`, sect. 3.1). Conséquence : le pas de temps ne résout **aucun système linéaire** — l'inversion de M est une division composante par composante dans le noyau de mise à jour (sect. 4.3). C'est ce qui rend la méthode *explicite* et massivement parallèle.

En contrepartie de la collocation, la **matrice de rigidité**

$$R^e_{ij}(b) = \int_{K^e} b \, \nabla\phi_i \cdot \nabla\phi_j$$

n'est pas diagonale — et FUnTiDES ne l'assemble jamais. Seule son **action** Rp (le *vecteur de rigidité*) est calculée, à la volée, à chaque pas de temps. C'est là que se concentre le coût de calcul, et c'est l'objet de la section suivante.

4.2 Le vecteur de rigidité : sum-factorization et GEMM

Sur un hexaèdre tensorisé d'ordre r, calculer Rp naïvement coûte $O\big((r+1)^6\big)$ opérations (matrice dense de taille $(r+1)^3 \times (r+1)^3$). La **sum-factorization** exploite la structure tensorisée pour descendre à $O\big(r\,(r+1)^3\big)$: au lieu d'un produit matrice-vecteur géant, une succession de produits par matrices de dérivation 1D dans chaque direction. Concrètement, `computeElasticStiffnessSumFact` (`Qk_Hexahedron_Lagrange_GaussLobatto.h`, ligne 653) procède en trois passes par élément :

1. **gradients de référence** du déplacement aux $(r+1)^3$ points GLL, par direction ξ, η, ζ ;
2. **par point** : inversion de la Jacobienne 3×3 de l'élément (transformation référence → physique), puis évaluation de la loi de comportement (sect. 3.2) produisant le flux de contrainte — injectée comme *callback* par point de quadrature, ce qui permet d'interchanger Iso/VTI/TTI sans dupliquer le squelette ;
3. **divergence transposée** des 9 flux de référence (F_ξ, F_η, F_ζ, chacun $3\times$ le nombre de nœuds) vers le vecteur résultat.

Il existe deux variantes parallèles de ce noyau (sect. 5.2) : l'une par équipe (`computeElasticStiffnessSumFactTeam`, ligne 686) et une spécialisation pour éléments à **Jacobienne constante** (ligne 703) qui met en cache les valeurs géométriques — sur un maillage cartésien régulier, toute la géométrie est identique d'un élément à l'autre, une optimisation classique du SEM.

Pour l'**acoustique uniquement**, FUnTiDES offre une alternative algébrique : la formulation *tensorielle* `Qk_Hexahedron_Tensorial_GEMM` (`src/discretization/fe/impl/tensorial/include/Qk_Hexahedron_Tensorial.h`, ligne 22), qui reformule l'action de rigidité scalaire en produits **matrice-matrice** denses — des *GEMM* (*GEneral Matrix-Matrix product*, nomenclature des bibliothèques BLAS : le produit $C = A \times B$ pour des matrices de tailles quelconques, l'opération la plus finement optimisée du calcul intensif) — délégués à `KokkosBlas` (`matmul_NN_team` /`matmul_TN_team`, lignes 588 et 601). Le dispatch se fait dans `computeElementContributions_Acoustic` (`sem_solver_impl.h`, lignes 342–347) : intégrale `TENSORIAL_GEMM` → chemin GEMM (ligne 224), intégrale `MAKUTU` (Gauss–Lobatto) → chemin plat (ligne 352). L'élastique, lui, n'a pas de chemin GEMM : ses 9 flux et sa loi de comportement par point restent en sum-factorization.

Leçon pour l'optimisation : " même opération mathématique, deux implémentations interchangeables (GEMM vs sum-factorization, flat vs team) " est un motif dominant de ce code. C'est exactement le genre de décision structurée qu'une extension d'optimisation se propose d'explorer systématiquement.

4.3 Intégration temporelle : leapfrog amorti

Après discrétisation spatiale, le système semi-discret (`formulation.md`, sect. 2, éq. 29) est intégré par un schéma **leapfrog** (Störmer– Verlet) d'ordre 2, avec le terme d'amortissement de bord traité de façon implicite pour la stabilité. Au nœud I, à chaque pas (`formulation.md` sect. 3.2) :

$$u_I^{n+1} = \frac{2\,M_I\,u_I^n - \left(M_I - \frac{dt}{2}S_I\right)u_I^{n-1} - dt^2\,F_I^n}{M_I + \frac{dt}{2}\,S_I}$$

et c'est **exactement** le calcul du noyau `updateFieldsForward` (`sem_solver_impl.h`, lignes 1388–1519) — ici pour l'acoustique :

Algorithme 5 Mise à jour leapfrog amortie — noyau acoustique

```
float next_val = (2.0f * mass_matrix(I) * current_field[0](I) -
                  (mass_matrix(I) - 0.5f * dt_local * damping_matrix[0](I)) * prev_field[0](I) -
                  dt2_local * work_vector[0](I));
...
prev_field[0](I) = next_val / (mass_matrix(I) + 0.5f * dt_local * damping_matrix[0](I));
prev_field[0](I) *= taper_coeff(I);        // éponge (cf. 3.4)
current_field[0](I) *= taper_coeff(I);
```

Points de lecture :

- **deux niveaux de temps suffisent** en direct : les tableaux `prev`/`curr` sont permutés par pointeur (swap) après chaque pas — pas de recopie des champs ;
- le noyau est **le même pour toutes les physiques**, avec des boucles internes sur `kNumFields` (1 en acoustique, 3 en élastique) ; la version élastique gère à part les nœuds de surface libre (sans amortissement) ;
- l'atténuation SLS (sect. 3.5) est **fusionnée dans le même noyau**, boucle sur les mécanismes `l` à l'intérieur du lambda ;
- l'éponge est appliquée ici aussi (multiplication par `taper_coeff`).

Stabilité CFL : comme tout schéma explicite, le pas de temps est borné par le temps de parcours le plus court entre deux points du maillage, $dt \lesssim C\,h/v_{\max}$, avec h la taille de maille et C une constante du schéma (d'autant plus contraignante que l'ordre r est élevé : les points GLL sont plus proches au bord de l'élément). C'est pourquoi le pas du harnais de référence est `kDt=0.001` s pour un domaine de 2000 m (sect. 6.2), et pourquoi " faire moins de pas de temps " n'est pas une optimisation disponible : on ne peut agir que sur le **coût d'un pas**.

Le mode **adjoint** (marche arrière) utilise une variante à **trois tampons** (`updateSolutionBackward`, `sem_solver_impl.h`, ligne 117 ; `swap()` à trois niveaux dans `wavefield_elastic.h`, lignes 111–135), nécessaire pour évaluer la dérivée seconde temporelle du champ adjoint sans recalcul.

4.4 Variantes DG et DG-SEM : pourquoi de la Galerkin discontinue

La continuité entre éléments (SEM continu) impose la synchronisation des nœuds partagés (sect. 5.3). La **Galerkin discontinue** (DG) lève cette contrainte : chaque élément porte ses propres degrés de liberté — pas de nœuds partagés — et le coupling passe par des **flux numériques aux faces**, comme en volumes finis. FUnTiDES utilise le pénalisateur **SIPG** (*symmetric interior penalty Galerkin*) : la discontinuité de champ et de gradient aux faces est pénalisée par un coefficient proportionnel à r^2/h, calculé par `computeSIPGPenaltyFromArea` (`dg_penalty.h` ; usage dans `dg_solver_impl.h`, lignes 299–301 et 380–381). Le noyau de faces `computeBoundaryDampingAndInterfaceFlux` (`dg_solver_impl.h`, ligne 196) traite **dans un seul `parallel_for`** les faces externes (amortissement) et les faces internes (flux d'interface), un exemple précoce de fusion de noyaux dans ce code.

Le solveur **DG-SEM** (`src/solver/fe/DG-SEm/`) couple les deux mondes dans un seul domaine : un **intérieur SEM** (efficace, où la solution est lisse) entouré d'une **couche DG** qui joue le rôle de frontière absorbante/troncature. Le découpage spatial est un simple seuil en z (`DgSemBoundaryZ`, cf. sect. 6.2). Le pas de temps est un schéma décalé (*staggered*) : contribution DG, contribution SEM, puis **mise à jour de Verlet fusionnée des deux sous-domaines dans un seul lancement de noyau** (`applyVerletFused`, `dg-sem_solver_impl.h`, ligne 273 ; lancement unique `"DG-SEMFusedVerlet"`, ligne 302 ; orchestration dans `computeOneStep`, ligne 337). C'est ce solveur, historiquement profilé et optimisé, qui sert de cas test au chapitre 6 — et son mode distribué est volontairement refusé en mono-étape (garde-fou ligne 345 : interdit d'appeler `computeOneStep` en mode distribué, il faut passer par la séquence `computeForces` → synchronisation → `updateSolution*`).

4.5 L'état adjoint et le gradient : le lien avec l'imagerie

Le chapitre 2 a motivé la FWI ; en voici la mécanique, résumée de `documentation/acoustic/formulation.md` sect. 4 (la référence fait la démonstration complète, que ce cours ne recopie pas ; pour le cadre général, voir Plessix, 2006).

Le gradient de J vis-à-vis du modèle ne peut pas être calculé par différences finies (une perturbation par paramètre = une propagation !). L'**état adjoint** le contourne : on définit le champ adjoint q comme la solution d'un problème de propagation **renversé en temps** — un problème en valeur finale, injecté aux récepteurs par le résidu $\mathcal{R}^*(\mathcal{R}p - d)$, et propageant l'information des données vers tout point du sous-sol. La structure de l'opérateur L de l'équation d'onde rend ce renversement remarquablement économique : les termes de masse et de rigidité sont **symétriques** en temps, seul le terme d'amortissement de bord change de signe ($L_3^* \neq L_3$, `formulation.md` sect. 5.2). Le même solveur, à un signe près, propage donc l'onde directe et l'onde adjointe.

Le gradient s'obtient alors par corrélation temporelle des deux champs :

$$\nabla J(\kappa, b) \;=\; -\begin{pmatrix} \displaystyle\sum_n \sum_{ij} \phi_i \phi_j \, \frac{q_i^{n-1} - 2q_i^n + q_i^{n+1}}{dt^2} \, p_j^n \\ \displaystyle\sum_n \sum_{ij} \nabla\phi_i \cdot \nabla\phi_j \; q_i^n \; p_j^n \end{pmatrix}$$

Implémentation : les noyaux d'accumulation vivent dans `src/gradient/impl/` (acoustique : `DifferentiatorAcoustic` ; élastique : `DifferentiatorElastic`), avec une distinction importante pour le parallélisme — modèle par **éléments** (stockage indexé par élément, pas de concurrence) vs par **nœuds** (nœuds partagés entre éléments, addition atomique `ATOMICADD` requise, cf. sect. 5.2). La référence complète du mécanisme, y compris le **préconditionnement** (division par le volume nodal Ω_I pour retrouver le noyau de sensibilité lisse — `formulation.md` sect. 6, dont l'anti-pattern " diviser par $M(\kappa)$ au lieu de Ω_I " biaise le gradient), est à lire directement dans ce document.

Chapitre 5 — HPC : Kokkos, GPU/CPU, MPI

Ce chapitre décrit **comment** les mathématiques du chapitre 4 tournent sur du matériel parallèle — et quels leviers une optimisation peut actionner sans toucher à la physique.

5.1 Kokkos : la couche de portabilité

Écrire un noyau deux fois (OpenMP pour CPU, CUDA pour GPU) double la surface de test et fait diverger les performances. FUnTiDES utilise **Kokkos** : chaque noyau s'écrit une fois comme un `Kokkos::parallel_for` avec un lambda portable (`KOKKOS_LAMBDA`), et le même code source compile vers les trois espaces d'exécution Serial, OpenMP et Cuda selon la configuration — le code ne contient **aucun** `#ifdef__CUDACC__` ou branche GPU/CPU explicite. La CI vérifie le chemin CPU (`.github/workflows/ci-cpu.yaml`) et des références de timing CPU et GPU sont stockées sous `tests/benchmarks/baselines/`.

Concepts Kokkos à connaître pour lire le code :

- `Kokkos::View` : le tableau multidimensionnel portable ; il porte à la fois l'espace mémoire (où vivent les données) et la disposition (`LayoutRight`, colonne-major-friendly pour les accès par nœud). Le type `arrayReal` du solveur est un `View` de `float`.
- **Espaces mémoire** : `HostSpace` (CPU) contre `CudaUVMSpace`/`CudaSpace` (GPU). L'exemple Python laisse le choix à l'utilisateur — `examples/fe/solver_cartesian.py` documente `MemSpace.CPU="HostSpace"` contre `MemSpace.GPU="CudaUVMSpace"` (mémoire unifiée, lignes 44–50), et `solver_cartesian_mpi.py` retombe sur `HostSpace` si l'installation Kokkos locale n'a pas de CUDA (lignes 47–50).
- **Transferts hôte ↔ device** : jamais d'écriture directe du côté hôte dans une vue device. Le motif canonique, visible dans le harnais du chapitre 6 : allouer la vue device, créer un miroir hôte (`Kokkos::create_mirror_view`), remplir le miroir, puis `Kokkos::deep_copy(mirrorâĘŠdevice)` (et `create_mirror_view_and_copy` dans l'autre sens pour relire un résultat).
- `Kokkos::fence()` : barrière de fin de noyaux, nécessaire quand le résultat d'un `parallel_for` est lu par le flot hôte ou par un noyau d'un autre espace ; le solveur l'encapsule dans la macro `FENCE` (`src/utils/include/sem_macros.h`, ligne 12), posée entre les phases de calcul (`sem_solver_impl.h`, lignes 82–126). Le **placement des fences** est un axe d'optimisation délicat : trop peu, les résultats sont erronés ou les flux concurrents se sérialisent ; trop, on perd du recouvrement. Les déplacements de fence ont une contrepartie importante : sur des timings chronométrés côté CPU, supprimer un fence peut sembler accélérer alors qu'il ne fait que rendre la mesure fausse — c'est une leçon des sessions d'optimisation passées sur ce code.

5.2 Deux politiques de parallélisation des éléments

Le calcul des contributions élémentaires (sect. 4.2) se parallélise **par élément**. FUnTiDES maintient deux variantes, dispatchées statiquement sur l'ordre (`sem_solver_impl.h`, lignes 731–736) :

- **variante** `_Flat` (ex. `computeElementContributions_Iso_Flat`, ligne 635) : `Kokkos::RangePolicy` — un fil d'exécution par élément, buffers de travail dans la pile du fil. Simple, mais chaque fil réserve les siens : rentable quand les buffers sont petits. C'est le chemin retenu pour l'élastique d'ordre 1 (`kMaxOrderForFlatElastic=1`, `sem_solver.h`, ligne 221).
- **variante** `_Team` (ex. `computeElementContributions_Iso_Team`, ligne 741) : `Kokkos::TeamPolicy` — **une équipe par élément**, les $(r+1)^3$ points GLL répartis dans l'équipe via `Kokkos::TeamThreadRange`, avec mémoire **scratch** partagée (champs locaux, flux, géométrie) et barrières internes là où les contributions se replient. C'est le chemin des ordres élevés, où la réutilisation de la scratch paie le coût de synchronisation.

Deux autres mécanismes traversent tous les noyaux :

- **Précision `float` (32 bits)** partout pour les champs et le modèle — le standard du domaine, où la donnée sismique ne justifie pas le double, et un doublement effectif de la bande passante mémoire.
- `ATOMICADD` pour l'assemblage aux nœuds partagés : en SEM continu, un nœud appartient à plusieurs éléments qui y ajoutent leur contribution simultanément (matrice de masse, matrice d'amortissement au sect. 3.4, gradients par modèle sur nœuds au sect. 4.5). Sur GPU c'est une addition atomique ; en modèle par éléments, au contraire, chaque élément écrit sa case sans concurrence — un choix de disposition qui élimine les atomiques.

5.3 MPI et décomposition de domaine

La parallélisation inter-nœuds est **séparée du solveur** : elle passe par une interface pluggable, `BoundarySynchronizer` (`src/solver/fe/impl/common/include/boundary_synchronizer.h`, ligne 36). Son backend MPI (`MPIBackend`, `mpi_backend.h`, ligne 19) fait les échanges non bloquants `MPI_Isend`/`MPI_Irecv` des zones fantômes. La séquence canonique d'un pas distribué est :

1. `computeForces` (noyaux élémentaires, sect. 5.2) ;
2. **pack** → **échange** → **accumulate** via le `BoundarySynchronizer` (appelé par l'orchestrateur *entre* `computeForces` et la mise à jour) ;
3. `updateSolutionForward` (leapfrog, sect. 4.3).

Deux choix à connaître :

- la **partition** n'est pas faite par un bibliothèque de partitionnement : c'est une **découpe 1D en slabs** calculée côté Python (`get_partition_1d`, `examples/fe/solver_cartesian_mpi.py`, ligne 65) — suffisant pour des maillages cartésiens, et représentatif de la physique (le split DG/SEM du sect. 4.4 est lui aussi une découpe en z) ;
- la **découpe MPI × GPU** : les exemples `examples/fe/solver_multigpu.py` et `solver_multigpu_multinode.py` montrent le mode multi-GPU (un rang par GPU) et multi-nœud.

La sémantique fine (assemblage de masse par morceaux, équation de bord) est documentée dans `documentation/acoustic/domain-decomposition.md`.

5.4 Le contexte cluster : les conditions réelles d'exécution

Les contraintes matérielles ci-dessous ne sont pas anecdotiques : elles conditionnent la validité de toute mesure d'optimisation (sect. 6.3).

- **Deux générations de GPU, deux préfixes Kokkos**. La TPL Kokkos installée (hors dépôt, à `/appli_RD/j1039830/local`) est compilée **uniquement pour l'architecture Blackwell `sm_103`**. Le nœud de mesure `elba-x02` (RTX 6000, `sm_120`) ne peut pas exécuter ces binaires ; `scripts/build_kokkos_sm120.sh` construit un second préfixe Kokkos, aux fonctionnalités identiques, pour `sm_120`. Le script d'exécution du harnais force `LD_LIBRARY_PATH` vers ce préfixe. **Leçon** : compiler pour la bonne architecture n'est pas un détail — un binaire `sm_103` ne tourne tout simplement pas sur `sm_120`.
- **glibc** : la pile (Python 3.11, cmake, CUDA) ne se construit et ne s'exécute **pas sur le nœud de login** (glibc trop ancienne) : la construction passe par `srun` vers le nœud de calcul, le script `scripts/build_dgsem_bench.sh` réinjecte lui-même sur le nœud si on le lance depuis l'extérieur.
- **Pas de `-O3` sur le DG-SEM** : le solveur contient un *use-after-free* latent (dans `SEMsolver::updateFieldsForward`) que `-O3` transforme en crash — l'état de référence validé est un build sans `CMAKE_BUILD_TYPE` (note `README.md`, ligne 147, et en-tête de `scripts/build_dgsem_bench.sh`). Une " optimisation " qui casse le build de référence n'en est pas une.
- **Mesurer sur un GPU au repos** : sur un nœud partagé, un timing peut varier d'un facteur 4–5 selon que le GPU est occupé. Le harnais épingle son GPU (`setenv("CUDA_VISIBLE_DEVICES","7",1)`, cf. sect. 6.2) et son script d'exécution (`scripts/dgsem_b300_exec.sh`) attend un GPU calme avant de mesurer. Un gain mesuré sur une machine bruyante n'est pas un gain.

Chapitre 6 — Le cas test et l'optimisation

Ce chapitre fixe le protocole d'évaluation des choix d'optimisation sur FUnTiDES.

6.1 Le harnais : un instrument, jamais le sujet

Le cas test est le fichier `tests/benchmarks/cpp/src/bench_solver_fe_cartesian_dgsem_acoustic.cc` (247 lignes). Sa raison d'être, énoncée dans son en-tête : être **l'instrument de mesure** de sessions d'optimisation, jamais leur sujet. Une optimisation qui modifierait le harnais pour améliorer la mesure est disqualifiée d'office ; le harnais doit rester le spectateur neutre du code qu'il chronomètre.

Sa configuration reproduit volontairement le driver de référence `sem_proxy.cc` (source Ricker placée comme `InitSource`, équivalent maillé du domaine) pour que ses temps soient comparables à l'état de référence.

6.2 Anatomie du cas test

Ce que le harnais simule — un solveur DG-SEM acoustique (sect. 4.4) sur un maillage cartésien **structuré** :

Algorithme 6 **Configuration du cas test DG-SEM**

```
constexpr int kOrder = 2;        // ordre polynomial r
constexpr int kEx = 24;          // 24 éléments par axe → 24³ éléments
constexpr float kDomainSize = 2000.0f;   // domaine cubique de 2000 m
constexpr float kDt = 0.001f;     // pas de temps (borné par la CFL, cf. 4.3)
constexpr int kNRhs = 1;         // une seule source
constexpr float kF0 = 5.0f;      // fréquence centrale Ricker
constexpr int kNDof = (kOrder + 1) * (kOrder + 1) * (kOrder + 1);  // 27
```

(lignes 48–56 ; `kEy`/`kEz` valent aussi 24). Le modèle est construit par `model::CartesianStructBuilder` avec `DgSemBoundaryZ=kDomainSize/2` (ligne 96) : moitié inférieure du domaine en DG, moitié supérieure en SEM. La source injecte uniquement dans la partie SEM, avec les poids de Lagrange de sect. 3.3 — jamais des poids uniformes, qui " excitent des modes non physiques et font diverger au pas CFL-réaliste " (commentaire du code, lignes 138–143).

Ce que le harnais mesure (bloc `PROXY_METRICS`, lignes 236–243) :

Métrique	Définition	Rôle
`time_s`	temps moyen par pas de temps (5 pas d'échauffement, puis boucle chronométrée de `n` pas)	**métrique principale** à minimiser
`mem_bytes`	empreinte mémoire persistante, calculée de façon déterministe (champs DG/SEM + buffers de travail, lignes 202–207)	métrique secondaire
`n_elem`, `n_node`, `kNDof`	tailles du problème	vérification de configuration
`output_file`	champ DG final en `float64` binaire	**preuve physique** : comparé à un champ de référence par normes `l2_rel`/`linf_rel`

Le protocole de mesure (lignes 209–223) : cinq itérations `computeOneStep` + `data.swapWavefields()` **hors chronomètre** (l'échauffement exclut le coût de lancement des premiers noyaux), puis la boucle identique sous chronomètre, divisée par `n` pour obtenir le temps par pas.

6.3 Le protocole ratchet : quand un changement compte

Le contrat d'une session d'optimisation sur ce harnais est un **ratchet** (cliquet) : une transformation du code n'est **acceptée** que si

1. la métrique s'améliore au-delà du bruit de mesure (médiane sur répétitions, marge minimale calibrée sur la variance observée — cf. l'échauffement et le GPU au repos du sect. 5.4) ;
2. **et** la physique est intacte : le champ final reste identique au champ de référence au sens des normes `l2_rel`/`linf_rel` — le harnais n'écrit pas son champ pour rien, c'est la clause de non-régression numérique.

Une transformation qui échoue (1) ou (2) est **rejetée et annulée** : on revient à l'état antérieur. C'est une exigence forte pour une extension MIMIR : quel que soit le formalisme (topos, C-sets…), chaque " choix d'optimisation " proposé devra démontrer son gain et son innocuité **par la mesure**, pas par construction — le ratchet est volontairement le seul juge.

Ordre de grandeur de référence : sur ce cas test (24^3, ordre 2), les sessions d'optimisation passées ont déjà réduit le temps par pas d'environ 16 % par des fusions de noyaux sur le membre de droite, et le profil par pas montre la part dominante prise par le noyau de faces DG (environ 60 % du temps). Les gains faciles sont pris ; un module de choix d'optimisation doit viser le reste.

6.4 Le paysage des axes d'optimisation

Les axes déjà explorés ou identifiés sur ce code — le vocabulaire avec lequel l'extension devra raisonner :

- **Fusion de noyaux** : un pas de temps déroule plusieurs `parallel_for` séparés (précalculs, contributions élémentaires, faces, mise à jour) ; les fusionner réduit les lancements et les lectures redondantes. Le DG-SEM " Fused Verlet " (sect. 4.4) est l'exemple abouti ; le même traitement est applicable aux précalculs du membre de droite.
- **Staging des données** : la série temporelle source (`rhsTerm`) est allouée sur device et lue à chaque pas ; le placement (device constant, scratch, rescopie) est un vrai choix de performance.
- **Élimination de recopies** : `deep_copy` hôte → device et champs tampons intermédiaires ; chaque copie évitable est de la bande passante récupérée.
- **Politique de lancement** : flat vs team (sect. 5.2), taille d'équipe, taille de scratch, `LaunchBounds` (`sem_solver_impl.h`, ligne 371) — bornes de bloc ajustées au registre occupé du noyau.
- **Compaction mémoire** : la disposition des `View` (ordres des dimensions, champs entrelacés vs planaires) gouverne la coalescence des accès ; le chemin GEMM vs sum-factorization (sect. 4.2) est aussi un choix de disposition.
- **Placement des fences** (sect. 5.1) — avec la mise en garde de mesure.

À côté de ces axes d'implémentation, une seconde famille : les **changements de méthode numérique**, qui modifient le schéma discret lui-même pour améliorer la précision du résultat — à runtime et mémoire gardés ou améliorés :

- **Ordre polynomial** r (p-convergence) : le coût par élément croît en $O(r(r+1)^3)$ (sect. 4.2), mais pour une précision donnée le nombre de points par longueur d'onde requis décroît avec l'ordre — le meilleur rapport précision/coût n'est pas nécessairement à r minimal ;
- **Intégrateur temporel** : leapfrog amorti (sect. 4.3) vs Verlet fusionné (sect. 4.4) ; plus loin, le *local time stepping* — un sous-pas là où la CFL est serrée — concentre l'effort de calcul là où il est utile ;
- **Conditions de bord** : éponge (sect. 3.4) vs C-PML (sect. 3.5) : absorption bien meilleure (moins de réflexions parasites, donc de meilleures images) au prix de variables de mémoire supplémentaires — un compromis précision/mémoire explicite ;
- **Physique enrichie** : atténuation viscoélastique SLS (sect. 3.5), anisotropie Iso→VTI→TTI (sect. 3.2) : rapprocher la simulation des données réelles, ce qui est la précision qui compte *pour l'imagerie*.

Les deux familles ne se jugent pas au même **oracle**. Les transformations d'implémentation gardent le champ de sortie identique au sens `l2_rel`/`linf_rel` — l'identité est l'invariant. Les changements de méthode, eux, déplacent le champ calculé *par construction* : leur invariant devient une qualité — la précision vis-à-vis d'une **référence convergée** (étude de convergence en r et en Δt, ou solution fabriquée), mesurée par les mêmes normes. La frontière entre les deux familles

est la tolérance : une perturbation sous le seuil (précision `float`) reste une transformation d'implémentation ; un changement de schéma dépasse le seuil *par intention*, et doit donc être jugé en précision. Le ratchet se généralise alors en compromis à deux objectifs : ne garder un changement de méthode que s'il améliore la précision à coût (`time_s`, `mem_bytes`) égal ou moindre — ou réciproquement.

Tous ces axes partagent alors une même structure : un **état de départ** (le code ou le schéma), des **transformations** (légales au sens de l'oracle choisi), un **coût à minimiser**, et des **contraintes** (mémoire, exactitude). C'est cette structure qu'une formalisation — par exemple catégorielle — peut modéliser : composants, dépendances de données, classes d'équivalence de transformations — sans que ce cours ne préjuge de sa forme.

6.5 Construire et exécuter le cas test

- **Construire** : `scripts/build_dgsem_bench.sh` (s'exécute sur le nœud de calcul, cf. sect. 5.4 ; construit le binaire du harnais avec le préfixe Kokkos adéquat).
- **Exécuter** : `scripts/dgsem_b300_exec.sh` — le script enregistré pour les mesures : il force le `LD_LIBRARY_PATH` du préfixe `sm_120`, épingle le GPU 7, attend que le GPU soit calme, et lance via `srun` sur le nœud de calcul (partition GPU RTX 6000). Le paramètre du harnais est un fichier de paramètres (`n=<pas>`, `output_file=...`).
- **Comparer** : le champ `output_file` se confronte au champ de référence par `l2_rel` et `linf_rel` — c'est la clause (2) du ratchet.
- **Sonder la variance** : `scripts/_probe_b300.sh` et `scripts/_variance_probe.sh` mesurent la dispersion des temps entre répétitions — la donnée qui calibre la marge minimale de gain crédible du sect. 6.3.

Références

Méthode numérique, propagation d'onde et imagerie — les fondements des chapitres 2 à 4.

1. Komatitsch, D., Barnes, C., Tromp, J. (2000). *Simulation of anisotropic wave propagation based upon a spectral element method.* Geophysics, 65(4) — la formulation pression–déplacement du couplage acousto-élastique, celle citée par `sem_solver_acoustoelastic.h`.
2. Komatitsch, D., Tromp, J. (2002). *Spectral-element simulations of global seismic wave propagation — I. Validation.* Geophysical Journal International, 149 — la référence canonique du SEM 3D sur hexaèdres GLL.
3. Komatitsch, D., Martin, R. (2007). *An unsplit convolutional perfectly matched layer improved at grazing incidence for the seismic wave equation.* Geophysics, 72(5) — le C-PML implémenté sur les branches `feature/pml-*-second-order` et documenté par `tests/units/solver/impl/test_pml_coefficients.cc`.
4. Martin, R., Komatitsch, D., Gedney, S. D. (2008). *A variational formulation of a stabilized unsplit convolutional perfectly matched layer for the isotropic or anisotropic seismic wave equation.* Computer Methods in Applied Mechanics and Engineering — la généralisation variationnelle du C-PML (contexte de la sect. 3.5).
5. Wang, T., Lee, K.-F., Teixeira, F. L. (2006) — récursion de convolution exacte pour le C-PML, telle que citée par `tests/units/solver/impl/test_pml_coefficients.cc`.
6. Virieux, J., Operto, S. (2009). *An overview of full-waveform inversion in exploration geophysics.* Geophysics, 74(6) — le cadre FWI du chapitre 2.
7. Plessix, R.-É. (2006). *A review of the adjoint-state method for computing the gradient of a functional with geophysical applications.* Geophysical Journal International, 167(2) — l'état adjoint du chapitre 4.

Note d'honnêteté bibliographique. L'environnement de rédaction n'avait pas d'accès sortant vers les bases bibliographiques (Crossref, OpenAlex, arXiv) : les entrées ci-dessus sont citées avec le niveau de détail vérifiable depuis le dépôt — les références 1, 3 et 5 sont ancrées par les fichiers du dépôt lui-même ; les références 2, 4, 6 et 7 sont canoniques du domaine. Les numéros de pages sont volontairement omis là où ils n'ont pas pu être contrôlés. Les publications de Besset, Citrain, Jacquet et Faucher n'ont pas pu être vérifiées et ne sont donc pas listées ici.

Annexe A — Définitions des termes techniques

Les termes sont rangés par catégorie, et alphabétiquement à l'intérieur de chaque catégorie. Chaque renvoi `Âğx.y` pointe vers la section où le terme est introduit dans le corps du cours. Les termes élémentaires — FLOP, DRAM, GEMM, etc. — sont dans la première catégorie ; les termes composés restent dans les catégories thématiques.

1. Vocabulaire de base : matériel et exécution

Terme	Définition
Bande passante	Débit de la mémoire en octets par seconde ; avec les FLOPs-crête, elle fixe le point de crête du matériel (sect. 1.3)
Cache	Petite mémoire rapide interposée entre les cœurs et la DRAM ; atténue le trafic DRAM quand les accès se répètent (sect. 1.3)
Coalescence (des accès)	Les fils d'un même groupe lisent des adresses contiguës, servies en une seule transaction mémoire ; dépend de la disposition des tableaux (sect. 1.3)
DRAM	*Dynamic Random-Access Memory* : mémoire principale du calculateur — des Go de capacité, l'étage le plus lent des niveaux accessibles au noyau ; son trafic est le dénominateur de l'IA (sect. 1.3)
FLOP /FLOPs	*FLoating-point OPeration* : une opération arithmétique en virgule flottante ; le travail d'un noyau se compte en FLOPs, la puissance d'un matériel en FLOP/s (sect. 1.3)
Fil (thread)	Unité d'exécution la plus fine ; des milliers de fils exécutent le même noyau, chacun sur ses données (sect. 1.6)
GEMM	*GEneral Matrix-Matrix product* (nomenclature BLAS) : produit dense $C = A \times B$ de matrices de tailles quelconques, l'opération la plus finement optimisée du calcul intensif ; ici, reformulation de la rigidité acoustique déléguée à `KokkosBlas` (sect. 4.2)
Hôte /device	Le CPU hôte orchestre — lance les noyaux, chronomètre — pendant que le device (le GPU) exécute ; d'où les artefacts de mesure côté hôte (sect. 1.7)
Lancement (kernel launch)	Invocation d'un noyau sur le device : coût fixe (mise en file) à chaque lancement, d'où l'intérêt de les fusionner (sect. 1.5)
Noyau (kernel)	Fonction exécutée massivement en parallèle par les fils du GPU ; l'unité de travail de l'optimisation (sect. 1.3)
Registre	Mémoire privée d'un fil, la plus rapide et la plus rare ; y garder des valeurs épargne la DRAM mais limite le nombre de fils résidents (sect. 1.3)
Équipe (team)	Groupe de fils partageant une mémoire scratch et une synchronisation ; l'unité du `TeamPolicy` de Kokkos (sect. 1.6)

2. Optimisation de code (chapitre 1)

Terme	Définition
Borné mémoire	Régime où le temps d'un noyau est fixé par le trafic DRAM, pas par le calcul (sect. 1.3)
Espace de recherche	L'ensemble des transformations légales candidates (sect. 1.1)
Fusion de noyaux	Réunir plusieurs lancements en un seul pour amortir le coût fixe et relire chaque donnée une fois (sect. 1.5)
Intensité arithmétique (IA)	FLOPs par octet déplacé en DRAM ; sous le point de crête, le noyau est borné mémoire (sect. 1.3)
Loi d'Amdahl	Le gain global est borné par la fraction du temps réellement accélérée (sect. 1.4)
Mapping parallèle	Répartition des données de travail sur les fils/équipes (flat vs team, scratch partagée) (sect. 1.6)
Occupation	Nombre de blocs résidents simultanément sur le GPU ; la fusion peut la réduire via la pression registre (sect. 1.5)

Terme	Définition
Optimisation illusoire	Transformation qui " améliore la métrique " en dégradant la mesure, pas le travail (ex. retrait de fences) — démasquée par le protocole (sect. 1.7)
Point de crête	Rapport FLOPs-crête /bande-passante-crête d'un matériel, de l'ordre de la dizaine de FLOP/octet sur les GPU de calcul (sect. 1.3)
Staging	Placement anticipé des données sur le device (préchargement de la série source, par ex.) pour éviter les recopies à chaque pas (sect. 6.4)
Toit de performance (roofline)	Modèle bornant la performance d'un noyau par min(FLOPs-crête, IA × bande-passante-crête) (sect. 1.3)
Transformation de méthode	Changement du schéma discret (ordre polynomial, intégrateur, bord, physique) qui améliore la précision à runtime et mémoire gardés ou améliorés ; jugée contre une référence convergée, non par identité (sect. 6.4)
Transformation légale	Modification du code qui préserve le champ de sortie (`l2_rel`/`linf_rel`), la mémoire et le build (sect. 1.1)

3. Mesure et protocole (chapitre 6)

Terme	Définition
Harnais (proxy)	Programme de mesure autonome qui exécute le code cible et imprime ses métriques (sect. 6.1)
`l2_rel /linf_rel`	Normes d'écart relatif L2 /L∞ entre champ calculé et champ de référence (sect. 6.2)
Ratchet (cliquet)	Protocole : un changement n'est gardé que s'il améliore la métrique sans régresser la physique (sect. 6.3)
`time_s /mem_bytes`	Métriques du harnais : temps moyen par pas de temps ; empreinte mémoire persistante (sect. 6.2)

4. Imagerie sismique (chapitre 2)

Terme	Définition
Born (linéarisation)	Linéarisation du problème direct autour d'un modèle — fondement de la sensibilité du gradient (sect. 2.3)
FWI	*Full-Waveform Inversion* : ajustement du modèle du sous-sol par minimisation de l'écart aux traces (sect. 2.2)
Tir (shot)	Une émission de source et l'enregistrement simultané par tous les récepteurs ; unité de travail indépendante de l'imagerie, d'où l'énorme parallélisme " un tir = une propagation " (sect. 2.2)
Trace sismique	Signal enregistré par un récepteur au cours du temps, pour un tir donné (sect. 2.2)

5. Propagation des ondes (physique) (chapitre 3)

Terme	Définition
Éponge (sponge)	Couche de bord où un profil d'atténuation multiplicatif amortit l'onde sortante (sect. 3.4)
Flottabilité b, module de compression κ	Paramétrisation acoustique $b = 1/\rho$, $\kappa = \rho v_p^2$; rend M et R linéaires en leurs paramètres (sect. 3.1)
Loi de comportement Iso/VTI/TTI	Contrainte–déplacement isotrope (Lamé), transverse isotrope verticale (6 coeff. Voigt), ou inclinée (matrice 6×6) (sect. 3.2)
Ondelette de Ricker	Source temporelle standard en sismique : dérivée seconde d'une Gaussienne (sect. 3.3)
PML /C-PML	*Perfectly Matched Layer* : couche absorbante par étirement de coordonnées ; absente de l'arbre de travail, seulement sur des branches (sect. 3.5)

Terme	Définition
SLS	*Standard Linear Solid* : atténuation viscoélastique par variables de mémoire à récurrence par pas (sect. 3.5)
Surface libre	Frontière où la pression s'annule (acoustique) /la contrainte est nulle (élastique) : le sol à l'air libre (sect. 3.4)
Voigt (notation)	Tenseurs symétriques 3×3 en vecteurs de 6 composantes, cisaillements " d'ingénieur " comptés doubles (sect. 3.2)

6. Méthode numérique (chapitre 4)

Terme	Définition
CFL	*Courant–Friedrichs–Lewy* : condition liant le pas de temps à la taille de maille et à la vitesse max (sect. 4.3)
DG	*Discontinuous Galerkin* : degrés de liberté par élément, couplage par flux de faces (sect. 4.4)
DG-SEM	Solveur couplé de FUnTiDES : couche DG absorbante autour d'un intérieur SEM, mise à jour de Verlet fusionnée (sect. 4.4)
État adjoint	Champ rétropropagé (problème en temps final) rendant le gradient calculable au coût d'une propagation (sect. 4.5)
GLL	*Gauss–Lobatto–Legendre* : points de quadrature colocalisés avec les nœuds d'interpolation, d'où masse diagonale (sect. 4.1)
Leapfrog (Störmer–Verlet)	Intégration temporelle explicite d'ordre 2 sur deux niveaux, à amortissement semi-implicite (sect. 4.3)
Préconditionnement du gradient	Division par le volume nodal Ω_I pour retrouver le noyau de sensibilité lisse (sect. 4.5)
SEM	*Spectral Element Method* : éléments finis d'ordre élevé sur base GLL ; matrice de masse diagonale (sect. 4.1)
SIPG	*Symmetric Interior Penalty Galerkin* : pénalisation symétrique des sauts de champ et de gradient aux faces DG (sect. 4.4)
Sum-factorization	Évaluation de l'action de rigidité en produits 1D tensorisés, coût $O(r(r+1)^3)$ au lieu de $O((r+1)^6)$ (sect. 4.2)

7. HPC : GPU, CPU, MPI (chapitre 5)

Terme	Définition
`ATOMICADD`	Addition atomique, pour l'assemblage concurrent aux nœuds partagés entre éléments (sect. 5.2)
Fence	Barrière d'achèvement des noyaux asynchrones (`Kokkos::fence()`) (sect. 5.1)
Kokkos	Bibliothèque de portabilité performante : un code source, plusieurs espaces d'exécution (Serial, OpenMP, Cuda) (sect. 5.1)
Scratch	Mémoire partagée d'équipe (cache), réutilisée entre fils d'une même équipe (sect. 5.2)
`sm_103 /sm_120`	Architectures NVIDIA Blackwell : une cible de compilation par génération de GPU du cluster (sect. 5.4)
UVM	*Unified Virtual Memory* : espace CUDA (`CudaUVMSpace`) visible depuis l'hôte et le device (sect. 5.1)
`View`	Tableau multidimensionnel Kokkos, portable en espace mémoire et en disposition (sect. 5.1)
Zone fantôme (ghost)	Copies des degrés de liberté de bord échangées entre rangs MPI à chaque pas (sect. 5.3)